

ALGORITHM DESIGN & ANALYSIS

The Complete Definitive Exam Blueprint & Solutions Guide

Question 1: Write DFS algorithm for an undirected graph

Core Concept: Depth-First Search (DFS) explores a graph by diving as deep as possible down a single path before backtracking. For an *undirected graph*, careful tracking of visited states is essential to prevent infinite loops across back-and-forth edges.

```
procedure DFS_Full(G):
    for each vertex  $u \in G.V$ :
         $u.color = WHITE$ 
         $u.\pi = NIL$ 
    time = 0
    for each vertex  $u \in G.V$ :
        if  $u.color == WHITE$ :
            DFS_Visit(G, u)

procedure DFS_Visit(G, u):
    time = time + 1
     $u.d = time$            // Discovery time
     $u.color = GRAY$        // Visited but unexplored neighbors

    for each  $v \in G.Adj[u]$ :
        if  $v.color == WHITE$ :
             $v.\pi = u$ 
            DFS_Visit(G, v)

     $u.color = BLACK$       // Completely explored
    time = time + 1
     $u.f = time$           // Finish time
```

Exam Twist & Intuition: Teachers love to ask about the **internal data structure** used by DFS. It is the **Stack (LIFO)**—implemented implicitly via recursion or explicitly. This LIFO property is exactly why it explores deep rather than wide. The runtime complexity is $O(V + E)$ because every vertex is transitioned through colors exactly once, and every edge in the adjacency list is scanned exactly once.

Question 2: Write and analyze the algorithm for printing a path from source to destination using attributes of nodes

Core Concept: After executing a traversal algorithm (like BFS or Dijkstra), the shortest path tree is encoded backwards using the parent pointer attribute (π). To print this path in the correct chronological order from source (s) to destination (v), we use recursion.

```
procedure Print_Path(G, s, v):  
    if v == s:  
        print s  
    else if v. $\pi$  == NIL:  
        print "No path from " + s + " to " + v + " exists."  
    else:  
        Print_Path(G, s, v. $\pi$ )  
        print v
```

Complexity Analysis:

- **Time Complexity: $O(V)$.** In the worst-case scenario (a simple linear chain), the path can pass through all vertices. The recursion depth matches the number of edges on the path, bound by V . Each activation record does $O(1)$ work.
- **Space Complexity: $O(V)$** due to the implicit recursion stack memory required to wind backwards to the source node before printing.

Question 3: Explain the Coin Change problem and write its greedy algorithm

Core Concept: Given an target amount W and an array of coin denominations, the objective is to minimize the absolute count of coins needed to sum exactly to W . The greedy heuristic dictates taking the largest denomination possible at every step.

```
procedure Greedy_Coin_Change(coins, W):
    sort(coins) in descending order
    coin_counts = empty map/list

    for each coin in coins:
        if W >= coin:
            count = W / coin    // Integer division
            coin_counts[coin] = count
            W = W % coin        // Remaining amount

    if W > 0:
        return "Exact change cannot be made greedily"
    return coin_counts
```

Why Greedy Fails (Guaranteed Conceptual Exam Topic): Greedy choice property holds **only for canonical coin systems** (like standard currencies where denominations grow such that each is roughly at least twice the size of the previous).

Counter-example: Let coins = {4, 3, 1}, and $W = 6$.

- **Greedy Choice:** Takes 4 first → remaining 2 → takes 1, 1. Total = **3 coins** (4 + 1 + 1).
- **Optimal Choice:** Takes 3 and 3. Total = **2 coins** (3 + 3).

Because a local optimal decision isolates us from global information, Greedy fails here; Dynamic Programming becomes strictly necessary.

Question 4 & 5: How to compute the length of LCS and obtain its entries

Core Concept: Longest Common Subsequence (LCS) finds the maximum sequence present in both strings in the original relative index order (not necessarily contiguous). Let string X have length m and string Y have length n .

Recurrence Relation Table Formulation:

$c[i, j] = 0$ if $i=0$ or $j=0$

$c[i, j] = c[i-1, j-1] + 1$ if $X[i] == Y[j]$

$c[i, j] = \max(c[i-1, j], c[i, j-1])$ if $X[i] != Y[j]$

Visual DP State

Execution Table

(Example: $X="ABC"$,
 $Y="AC"$)

-	$\emptyset$	A	C
$\emptyset$	0	0	0
A	0	1	1
B	0	1	1
C	0	1	2

```
// Length Extraction Algorithm
procedure LCS_Length(X, Y):
    m = X.length, n = Y.length
    let c[0..m, 0..n] be a new table
    let b[1..m, 1..n] be a pointer direction table

    for i = 0 to m: c[i,0] = 0
    for j = 0 to n: c[0,j] = 0

    for i = 1 to m:
        for j = 1 to n:
            if X[i] == Y[j]:
                c[i,j] = c[i-1,j-1] + 1
                b[i,j] = "DIAGONAL"
            else if c[i-1,j] >= c[i,j-1]:
                c[i,j] = c[i-1,j]
                b[i,j] = "UP"
            else:
```

```
        c[i,j] = c[i,j-1]
        b[i,j] = "LEFT"
    return c, b
```

Obtaining Entries (Backtracking from bottom-right):

```
procedure Print_LCS(b, X, i, j):
    if i == 0 or j == 0:
        return
    if b[i,j] == "DIAGONAL":
        Print_LCS(b, X, i-1, j-1)
        print X[i]
    else if b[i,j] == "UP":
        Print_LCS(b, X, i-1, j)
    else:
        Print_LCS(b, X, i, j-1)
```

Question 6: Solve problems in $O(\log n)$ time (The Binary Search paradigm)

Core Concept: To solve search queries in logarithmic time $O(\log n)$, you must continually partition your input space into equal halves. This is fundamentally achieved via a sorted database layout using continuous array index tracking.

```
procedure Binary_Search_Safe(arr, target):
    low = 0
    high = arr.length - 1

    while low <= high:
        // Safe from overflow bugs!
        mid = low + (high - low) / 2

        if arr[mid] == target:
            return mid
        else if arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Crucial Exam Trap Warning: Teachers look specifically for your formula for finding the middle index. Writing $mid = (low + high) / 2$ is mathematically correct but a **critical engineering vulnerability**. If $low + high$ exceeds the maximum storage allocation of a signed 32-bit integer, it wraps around negatively and triggers fatal execution boundary violations. Always write $mid = low + (high - low) / 2$.

Question 7: Analyze Core Algorithms (Rod Cutting, Levenshtein Edit Distance, Dijkstra, Prim's, Continuous Knapsack)

Below is a centralized summary of these essential architectural patterns:

Algorithm Name	Paradigm	Recurrence / Core Logic	Runtime Complexity
Rod Cutting	DP (Tabulation)	$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$	$O(n^2)$
Levenshtein Distance	DP (Matrix)	$\min(\text{ins}, \text{del}, \text{subst}) + 1$	$O(m \times n)$
Dijkstra	Greedy	Relaxation: $d[v] = \min(d[v], d[u] + w(u,v))$	$O((V+E) \log V)$
Prim's	Greedy	Select min cut-crossing edge connecting to tree	$O((V+E) \log V)$
Continuous Knapsack	Greedy	Sort by density ratio: $\text{value} / \text{weight}$	$O(n \log n)$ (sorting bound)

Question 8: Explain Binary Search algorithm in all aspects covered in this course

To fully answer an "all aspects" essay question, you must demonstrate loop correctness property alongside algorithmic mechanics:

- **Loop Invariant Property:** At the initialization of each loop cycle, the search target element, if present anywhere within the original immutable dataset, is guaranteed to be contained purely within the index range bounded by *[low, high]*.
- **Recurrence Equation Proof:** The search domain decreases by $1/2$ on each step. The relation can be formatted as:

$$T(n) = T(n/2) + \Theta(1)$$
 Applying Case 2 of the **Master Theorem** ($a=1, b=2, c=0 \implies \log_2(1) = 0$), the execution time collapses cleanly to $\Theta(\log n)$.

Question 9: What are the three classes of problems, and why is the satisfiability (SAT) problem significant?

The field of complexity splits computational problems into three primary spaces:

1. **P (Polynomial Time):** Solvable efficiently in polynomial time $O(n^k)$ on a deterministic machine (e.g., Shortest Path, Sorting).
2. **NP (Nondeterministic Polynomial):** Solutions cannot necessarily be found efficiently, but any proposed candidate verification assignment can be verified as correct or incorrect in polynomial time.
3. **NP-Complete:** The absolute hardest architectural problems within the NP class. If any individual NP-Complete problem can be unlocked using a deterministic polynomial time system, then *every single problem inside the entire NP spectrum* is instantly cracked simultaneously.

Significance of the Satisfiability (SAT) Problem: SAT was explicitly proven by the **Cook-Levin Theorem** to be the world's **first ever NP-Complete problem**. It serves as the foundation for modern complexity theory because all other NP-Complete proofs are completed by reduction from SAT or its variants (like 3-SAT).

Question 10: What are the elements of a greedy approach?

For a problem to be solved optimally via a greedy architecture, it must possess two strict structural elements:

1. **Greedy Choice Property:** A globally optimal state tracking solution can be achieved by choosing the absolute best local candidate configuration at each progressive step without ever reviewing past decisions.
2. **Optimal Substructure:** An optimal overarching solution to the main algorithmic structure contains within itself the strictly optimal localized configurations to all its sub-components.

Question 11: What are the two ways the inefficiency added by overlapping subproblems can be addressed?

When a recursive problem branches repeatedly into identical processing evaluations, it produces exponential computational waste. We eliminate this duplication in two ways:

Strategy	Direction	Implementation Mechanic
Memoization	Top-Down	Maintains traditional recursive structure but intercepts the entry code to check an associative hash table array. Returns instantly if solved.
Tabulation	Bottom-Up	Eliminates recursion stack frames completely. Iteratively satisfies base cases first, systematically populating an iterative table matrix.

Question 12: What is the approximate solution of the knapsack problem?

The 0/1 Knapsack problem is fundamentally NP-Hard. To circumvent exponential runtime, an **Approximation Algorithm** relaxes the hard binary rule using a greedy ratio strategy:

1. Compute the value-to-weight density ratio v_i / w_i for all items.
2. Sort all items in strict descending order of this ratio metric.
3. Pack full items sequentially until the next item cannot fit completely.
4. Take the remaining capacity fraction of that boundary item (Fractional relaxation) or pick the max of the items chosen vs. the single heaviest item to ensure a **1/2-approximation bound**. This runs in tight $O(n \log n)$ time.

Question 13: Execute BFS on a graph (when needed, scan vertices from top to bottom and edges from left to right)

Core Execution Pattern: Breadth-First Search (BFS) processes nodes level-by-level using a FIFO Queue data structure. To correctly trace a graph exam query:

- Initialize all vertices to a WHITE state with distance infinity.
- Set source node to GRAY and distance 0, then push to the Queue.
- While the Queue is occupied, pop the head element u , examine all adjacent nodes in alphabetical/topological order; color them GRAY, set parent $\pi[v] = u$, update distance $d[v] = d[u] + 1$, and push them to the tail. Finally, color u BLACK.

Question 14: Compute keys and π for nodes of a graph after running PRIMS algorithm on it

During the generation of a Minimum Spanning Tree via Prim's algorithm, two parameters are continually updated across the node set:

- **key[v]:** Stores the absolute minimum edge weight connecting vertex v to any vertex already assimilated into the growing MST tree network. If no edge exists, it remains initialized to infinity.
- **$\pi[v]$:** Identifies the explicit structural parent vertex of node v which supplies that minimal cut edge weight.

Question 15: Demonstrate the maximum sub-array sum problem on a suitable problem instance

Problem Instance Example: Let the array be $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$.

Using **Kadane's Algorithm**, we dynamically track the running sub-array condition:

Index (i)	Element A[i]	Current Max Ending Here	Global Max Found So Far
0	-2	-2	-2
1	1	$\max(1, -2+1) = 1$	1
2	-3	$\max(-3, 1-3) = -2$	1
3	4	$\max(4, -2+4) = 4$	4
4	-1	$\max(-1, 4-1) = 3$	4
5	2	$\max(2, 3+2) = 5$	5
6	1	$\max(1, 5+1) = 6$	6 (Sub-array: [4, -1, 2, 1])

Question 16: Edit Distance problem (DP) and Code Generation (Greedy)

This dual question contrasts optimization strategies:

- **Edit Distance (DP):** Builds a structural character alignment matrix mapping the absolute minimal path of transformations. Every insertion, deletion, or replacement calculation references the optimal values of smaller adjacent subproblem cells.
- **Code Generation (Greedy - Huffman Codes):** To minimize the bit-length of generated transmission code, the algorithm takes a greedy approach: it identifies the two lowest frequency characters, combines them into a tree node, and repeats until a single structural root is built. This local reduction guarantees global character code minimization.

Question 17: How single source and all pair shortest path algorithms are related to each other

The Core Structural Connection: Single-Source Shortest Path (SSSP) algorithms (like Dijkstra or Bellman-Ford) compute shortest paths from a single fixed origin point to all other vertices. All-Pairs Shortest Path (APSP) algorithms (like Floyd-Warshall) compute shortest paths between *every possible pair of vertices* in the graph.

Algorithmic Relationship: You can construct an APSP solution by executing an SSSP algorithm repeatedly, iterating over every vertex as the source. For example, running Dijkstra V times yields a total runtime of $O(V \cdot (V + E) \log V)$, which is highly efficient on sparse layouts compared to default matrix multi-stage processing.

Question 18: Differentiate Divide & Conquer vs DP, and DP vs Greedy

Comparison Plane	Divide & Conquer	Dynamic Programming	Greedy Method
Subproblem Overlap	Strictly independent subproblems; no overlap.	Highly overlapping, recurrent subproblems.	Subproblems are not fully generated; immediate reduction.
Memory Cache	No caching; redundant paths re-evaluate.	Saves state globally in table structures.	Requires no state-saving historical matrix.
Decision Framework	Combines downstream structural results.	Evaluates all paths before finalizing values.	Commits to local optimum instantly.

Question 19: Why optimization problems are of special concern

Optimization problems seek the absolute best configuration from a vast space of valid options (maximizing profit, minimizing latency, etc.). They are uniquely challenging because a minor increase in input elements often causes the space of valid options to explode exponentially.

As a result, naive brute-force testing becomes physically impossible. They demand advanced algorithmic frameworks like DP or Greedy heuristics to isolate optimal solutions without exhausting physical computing resources.

Question 20: How the three types of optimization problem w.r.t. discussion in class for DP, differ from each other

Within dynamic programming theory, optimization problems are categorized based on their internal structural bounds:

1. **Matrix Chain Multiplication / Parenthesization Type:** Order-dependent problems where the choice of split points alters downstream processing configurations.
2. **Selection / Knapsack Type:** Binary choice configurations where you evaluate whether to include or exclude an element at each step, directly reducing the remaining structural resource capacity.
3. **Sequence Alignment / Editing Type:** Multi-choice grid systems (like LCS/Edit Distance) where you evaluate index-to-index matches against character alterations across multiple strings.

Question 21: What methodology is used to prove correctness in graph algorithms: loop invariants or something other?

While basic iterative array algorithms rely purely on standard **Loop Invariants**, graph structures require a broader set of proof methodologies due to their topological complexity:

- **Mathematical Induction:** Proving that if a structural state condition holds true for a path of length k , it remains correct when extended to length $k+1$.
- **The Cut Property:** Used explicitly to prove minimum spanning tree algorithms (like Prim's or Kruskal's). It states that for any cut of a graph, the minimum weight edge crossing that cut must belong to the MST.
- **Path Relaxation Properties:** Used in shortest path algorithms (like Dijkstra's) to prove mathematically that distance estimates converge monotonically toward their true minimal values.

Question 22: What is the issue with negative weight cycles in graphs?

The Core Failure Mechanic: A negative weight cycle is a closed loop in a graph whose total combined edge weight is less than zero. If an algorithm attempts to calculate the shortest path through such a loop, it can traverse the cycle infinitely to lower the total path cost.

Consequently, the true shortest path becomes completely **undefined** (*-inf*). Greedy approaches like Dijkstra will fall into infinite updates or output corrupted tables. This makes special algorithmic safeguards (like the edge-relaxation constraints in the Bellman-Ford algorithm) mandatory to detect and halt execution.